

Security Assessment Report

Automated Vulnerability Discovery

Target Infrastructure: http://testfire.net
Date of Assessment: 2026-05-20 03:21 IST
Reference ID: d06d2efb-ca2d-4e3d-b592-994b47f0f07c

1. Executive Summary

This document presents the findings of an automated security assessment conducted against **http://testfire.net** on 2026-05-20 03:21 IST. The assessment utilized the SmartFuzz testing engine to evaluate the target against a predefined set of vulnerability classes. The overall risk posture of the application is currently evaluated as **Critical**.

Assessment Metric	Value
Total Payloads Generated	236
Total Findings Confirmed	4
Critical Risk Findings	2
High Risk Findings	1
Medium Risk Findings	1
Maximum CVSS v3.1 Score	9.1
Average CVSS v3.1 Score	7.1

2. Authorization & Consent

Before this assessment was launched, the SmartFuzz operator was presented with a mandatory legal consent dialog. The scan only proceeded once the operator explicitly confirmed each of the following statements.

Confirmation Statements

1. I own this target, or I have written authorization from the owner.
2. I understand this scan will send potentially malicious payloads.
3. I accept full legal responsibility for the consequences of this scan.

Legal Reference — Information Technology Act, 2000

Unauthorized access to a computer resource, including the unauthorized scanning of computer systems for vulnerabilities, constitutes a punishable offence under the Information Technology Act, 2000 (India), **Section 43** (penalty for damage to computer, computer system, etc.) and **Section 66** (computer-related offences). Comparable statutes exist in most jurisdictions worldwide — notably the Computer Fraud and Abuse Act (United States), the Computer Misuse Act 1990 (United Kingdom), and the Council of Europe Convention on Cybercrime. SmartFuzz records the operator's authorization before each scan to ensure all testing activity is performed in compliance with applicable law.

3. Findings Summary

ID	Vulnerability	Parameter	Severity	CVSS
1	SSRF	content	Critical	9.1
2	SSRF	content	Critical	9.1
3	XSS	content	High	6.1
4	XSS	content	Medium	4.1

4. Detailed Findings Analysis

Finding 1: SSRF in parameter 'content'

Severity:	Critical
CVSS v3.1 Score:	9.1
CVSS Vector:	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:L
Target URL:	http://testfire.net/index.jsp?content=http%3A%2F%2F169.254.169.254%2Flatest%2Fmeta-data%2F
Method:	GET
Status Code:	200
Response Time:	0.25 seconds
Injected Payload:	http://169.254.169.254/latest/meta-data/
Evidence:	<pre><!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" > <head> <title>Altoro Mutual</title> <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" /> <link href="/style.css" rel="stylesheet</pre>

Finding 2: SSRF in parameter 'content'

Severity:	Critical
CVSS v3.1 Score:	9.1
CVSS Vector:	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:L
Target URL:	http://testfire.net/index.jsp?content=http%3A%2F%2F169.254.169.254%2Flatest%2Fuser-data%2F
Method:	GET
Status Code:	200
Response Time:	0.328 seconds
Injected Payload:	http://169.254.169.254/latest/user-data/

Evidence:

```
<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
<head> <title>Altoro Mutual</title> <meta
http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" /> <link href="/style.css"
rel="stylesheet
```

Finding 3: XSS in parameter 'content'

Severity: High

CVSS v3.1 Score: 6.1

CVSS Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

Target URL: <http://testfire.net/index.jsp?content=%3Cdetails+open+ontoggle%3Dalert%28%27XSS%27%29%3E>

Method: GET

Status Code: 200

Response Time: 0.281 seconds

Injected Payload: <details open ontoggle=alert('XSS')>

Evidence:

```
<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
<head> <title>Altoro Mutual</title> <meta
http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" /> <link href="/style.css"
rel="stylesheet
```

Finding 4: XSS in parameter 'content'

Severity: Medium

CVSS v3.1 Score: 4.1

CVSS Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:N/A:N

Target URL: <http://testfire.net/index.jsp?content=%27%3Balert%28%27XSS%27%29%2F%2F>

Method: GET

Status Code: 200

Response Time: 0.265 seconds

Injected Payload: `' ;alert('XSS')//`

Evidence:

```
<!-- BEGIN HEADER --> <!DOCTYPE html PUBLIC "-//W3C//DTD
XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
<head> <title>Altoro Mutual</title> <meta
http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" /> <link href="/style.css"
rel="stylesheet
```

5. Remediation Guidance

SSRF

1. Implement a strict allowlist of permitted destination domains, IP addresses, and protocols for any outbound server requests.
2. Actively block and reject network requests targeting internal, loopback, or reserved IP ranges (e.g., 10.0.0.0/8, 169.254.169.254, 127.0.0.1).
3. Disable HTTP redirection for server-side requests, or strictly re-validate the target URL against the allowlist after following any redirect.
4. Route all outbound application traffic through a dedicated egress proxy that explicitly drops internal network access.
5. Reference: OWASP Top 10 A10:2021 — Server-Side Request Forgery

XSS

1. Contextually output-encode all user-supplied data before rendering it in the browser (HTML, JavaScript, CSS, or URL contexts).
2. Implement a strict, restrictive Content-Security-Policy (CSP) HTTP response header to prevent the execution of inline scripts and untrusted sources.
3. Set the 'HttpOnly' and 'Secure' flags on all sensitive cookies (e.g., session tokens) to prevent theft via client-side scripts.
4. Sanitize any rich-text (HTML) input on the server side using a robust, actively maintained library (e.g., DOMPurify or Bleach).
5. Utilize modern frontend frameworks (React, Angular, Vue) that safely bind data to the DOM by default.
6. Reference: OWASP Top 10 A03:2021 — Injection

Disclaimer

This report was generated automatically. Results should be reviewed and validated by a qualified security professional. False positives may be present.